Automatic QA for LLM-powered System

Alisa Dammer, May 2026

About Me

Freelance MLE/MLOps/AIE/CTO by day, hobbyist learner and programmer by night. Love reading about new health-related researches. Love playing around with gadgets and tech in general.

Wanna connect?

@FatDataUnicorn@fosstodon.org

@FatDataUnicorn

Alisa Dammer#5584

alisadammer

alisadammer.com



Structure

1. Project context
2. Metrics hierarchy
    a. Deterministic
    b. Statistical
    c. Complex
3. Cost
4. Improvements area
5. QA

How and when do you trust LLM-output?

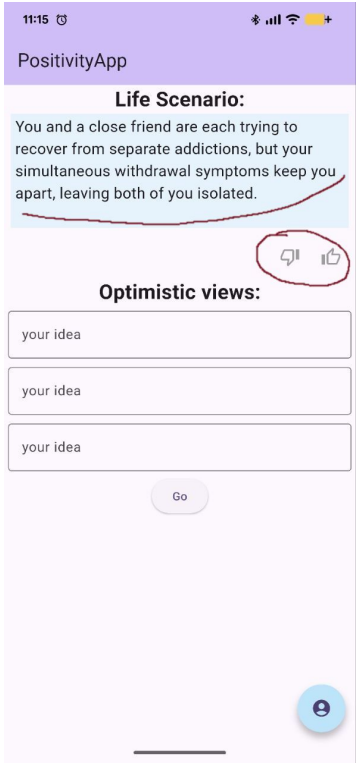
Mindset Training

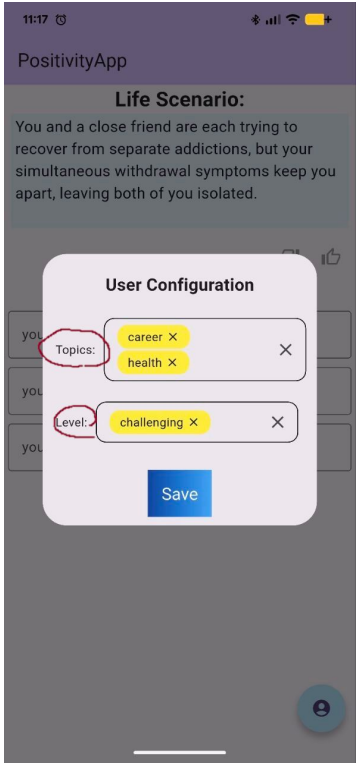
Harder, Better, Faster, Stronger!

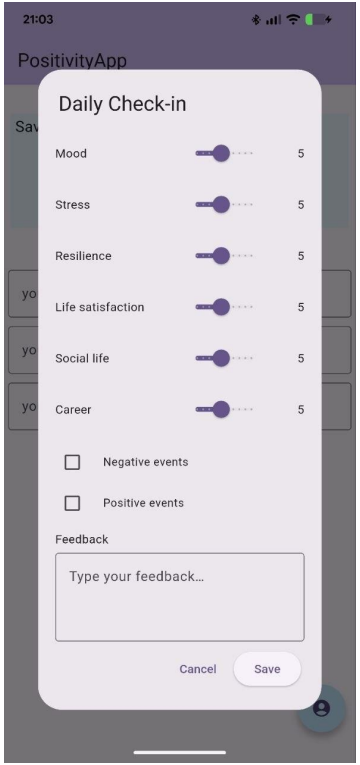
c DaftPunk

- User satisfaction
- Technical novelty
- Contextual match
- Error rate
- Cost

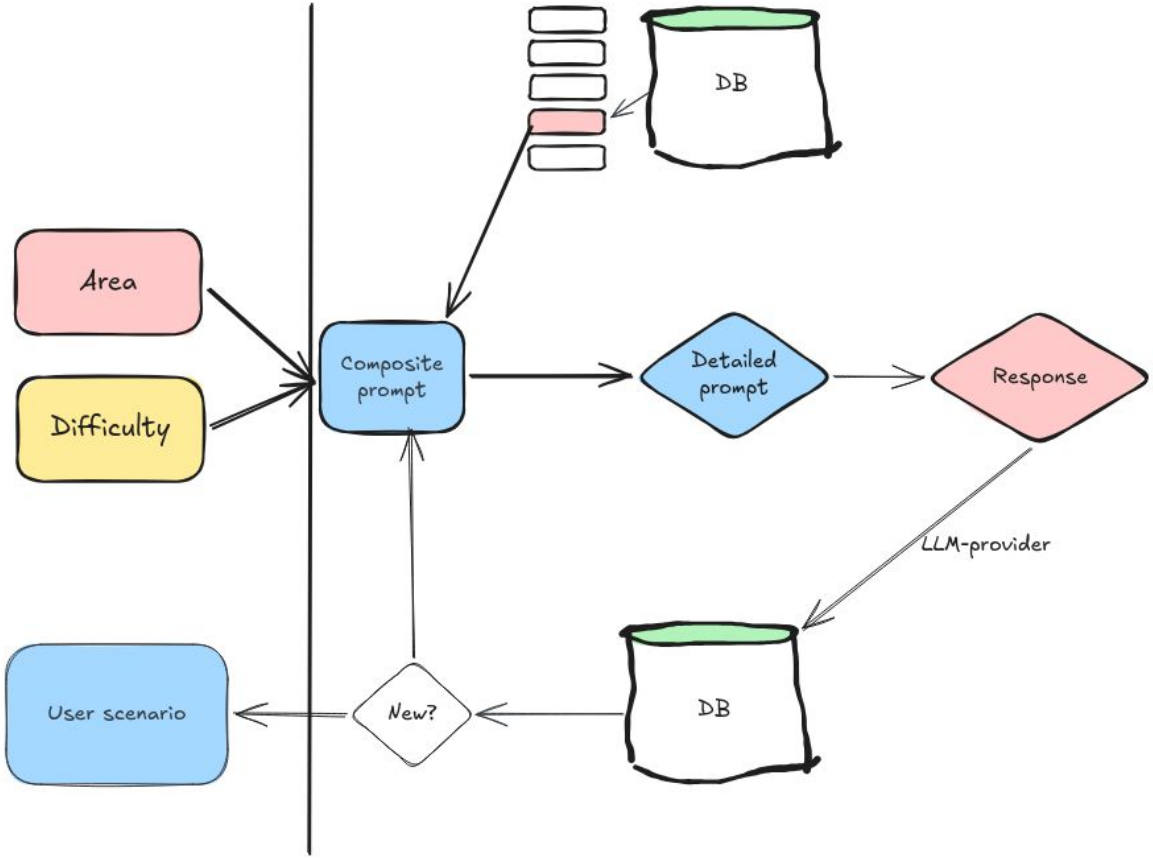
App structure



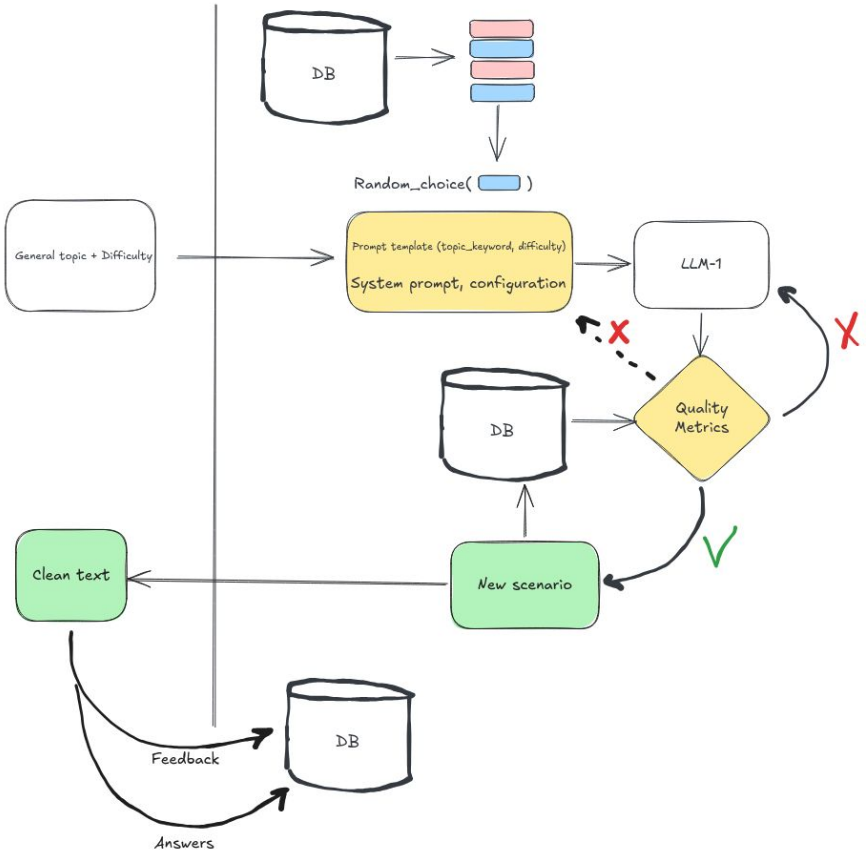




App structure



Prompt creation



What Are you looking at?

Deterministic metrics

- Keyword coverage
- Duplicates ratio
- Jaccard similarity

Key Coverage




```python
import re

def keyword_coverage(text, keywords):
    tokens = set(re.findall(r"\b\w+\b", text.lower()))
    matched = sum(1 for kw in keywords if kw.lower() in tokens)
    return matched / len(keywords) if keywords else 0.0
```

- Predefined keywords
- No synonyms
- Relevance only

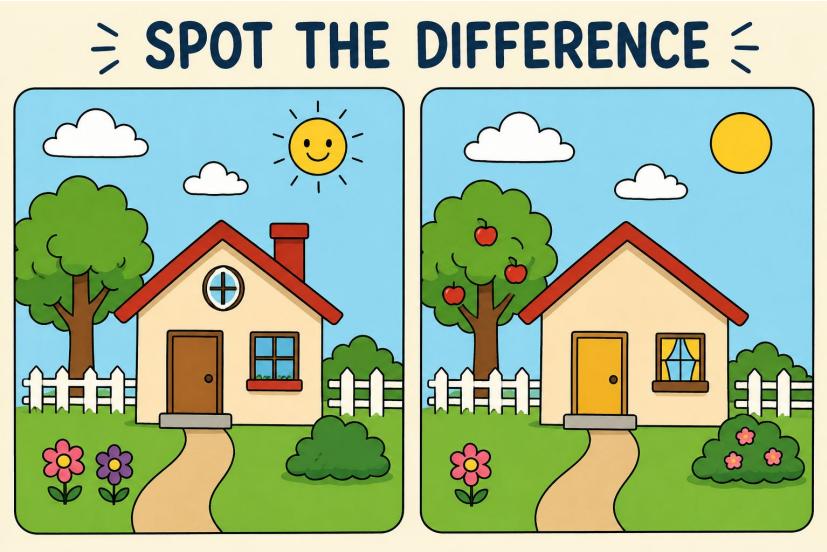
Duplicates Ratio




```python
def duplicate_ratio(outputs):
    total = len(outputs)
    unique = len(set(outputs))
    return (total - unique) / total if total else 0.0
```

- Low usability
- Precise match

Jaccard Similarity




```python
def tokenize(text):
    return set(re.findall(r"\b\w+\b", text.lower()))

def jaccard_similarity(text_a, text_b):
    a = tokenize(text_a)
    b = tokenize(text_b)

    intersection = len(a & b)
    union = len(a | b)

    return intersection / union if union else 0.0
```

- No semantic awareness
- Very limited usability

Statistical metrics

- self-BLEU
- Embedding Cosine Similarity (Sentence Transformers)

Self-BLEU



```python
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

def self_bleu(texts):
    smoothie = SmoothingFunction().method1
    scores = []

    tokenized = [text.lower().split() for text in texts]

    for i, candidate in enumerate(tokenized):
        references = [t for j, t in enumerate(tokenized) if j != i]
        score = sentence_bleu(
            references,
            candidate,
            smoothing_function=smoothie
        )
        scores.append(score)

    return sum(scores) / len(scores) if scores else 0.0
```

- Limited usability
- Prone to overestimation in cross-topic-vocabulary
- Computationally expensive

Embedding cosine similarity




```python
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

# Load a lightweight embedding model
model = SentenceTransformer("all-MiniLM-L6-v2")

text1 = "Your manager delays your promotion."
text2 = "Your boss postpones your advancement."

# Convert texts to vectors
embeddings = model.encode([text1, text2])

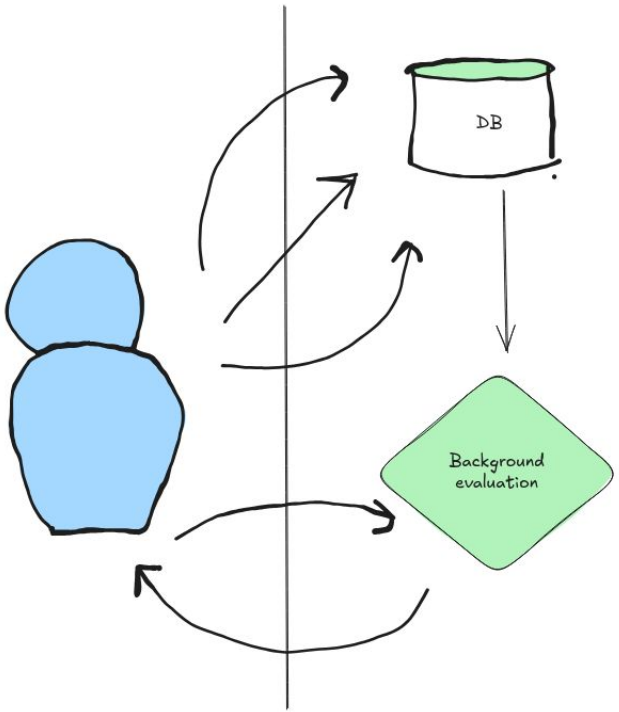
# Compute cosine similarity
score = cosine_similarity(
    [embeddings[0]],
    [embeddings[1]]
)[0][0]
```

- Expensive
- Domain expertise required

Complex and expensive

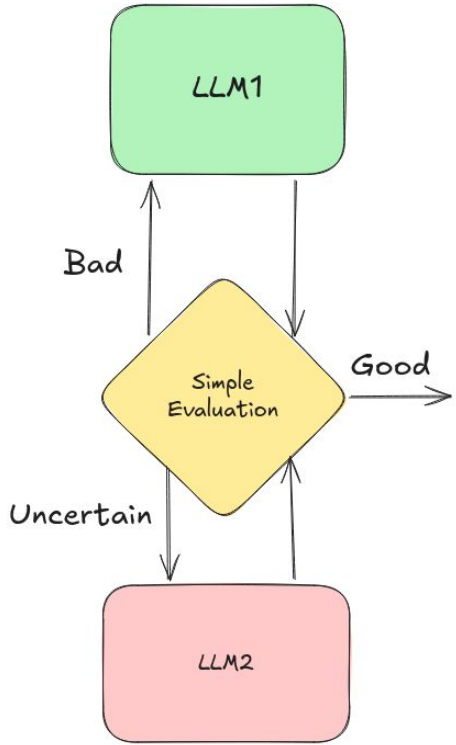
- Human evaluation
- Supervised custom models
- LLM-as-a-judge

Human eval



- Biased-prone
- Highest complexity
- Separate business process

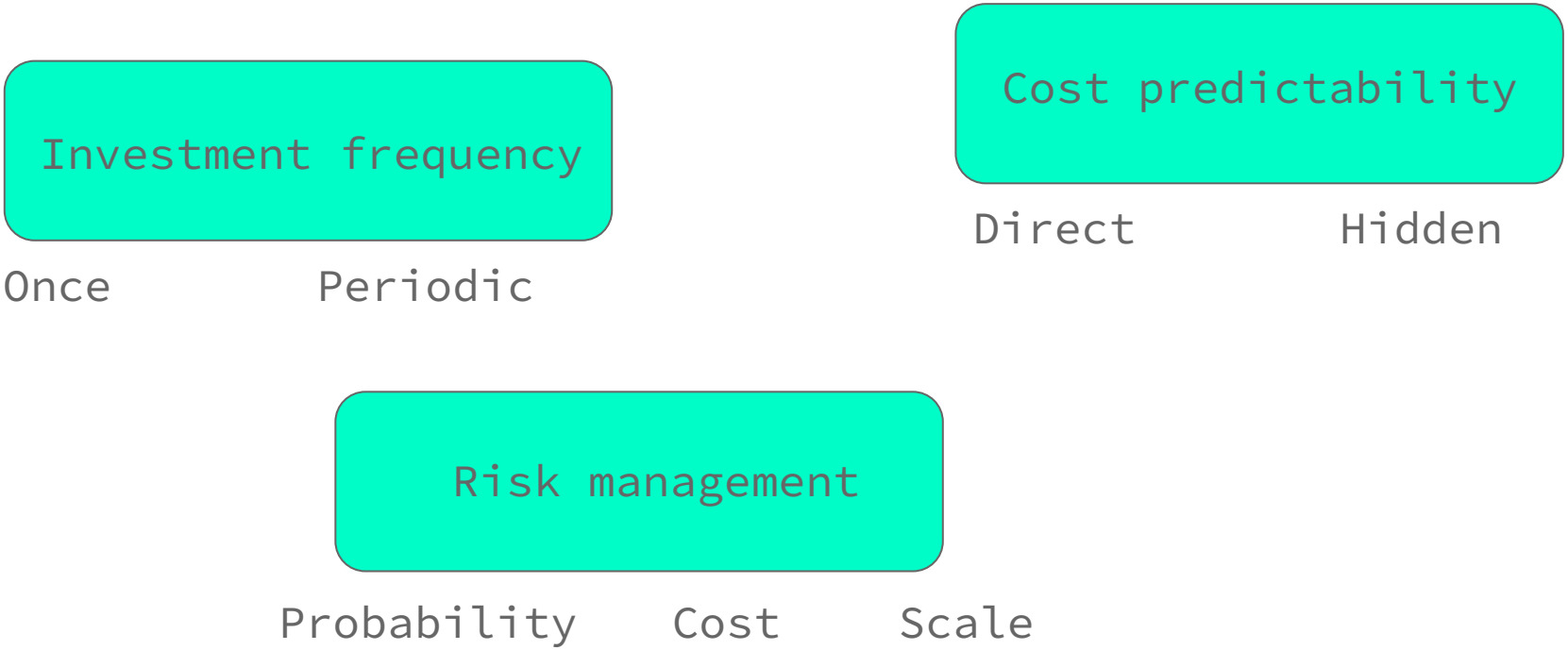
LLM-AS-A-JUDGE



- High complexity
- High cost
- Separate QA system

How much is the fish???!!!

Cost factors



Cheat Sheet

Metric	Purpose	Complexity	Cost	Stability	Signal strength
Keyword Coverage	Topic relevance	Low	Low	High	Mid
Duplicate Rate	Repetition validation	Low	Low	High	High
Jaccard Similarity	Lexical similarity	Low	Low	High	High
Self-BLEU	Diversity	Mid	Low	High	Mid/High
Emotional trend	Quality per dimension	High	High	Mid	High
Sentence Transformers	Diversity	Mid	Mid	Mid/High	Mid
Secondary LLM	Quality per dimension	High	Extremely High	High/Mid	High
…					

Anything else?

User Metrics

Long-term

- Goals scoring -> trend detection
- Free-text feedback -> Sentiment analysis

Psychological model efficiency

Short-term

- Like-button -> binary satisfaction/anomaly detection
- Re-generate -> negative signal for prompt

Generation quality score

Pre- vs Post-processing

- Include “simple” metric into generation logic
- Do not rely on external providers, if you have a choice
- Invest in data-driven UX/UI (extensive **deliberate** tracking)
- Invest in user analysis

Resources

- Jiawei Gu, Xuhui Jiang, et.al., “A Survey on LLM-as-a-Judge”, arXiv:2411.15594 [cs.CL]
- Michał Karp Anna Kubaszewska, et.al., “LLM-as-a-Judge is Bad, Based on AI Attempting the Exam Qualifying for the Member of the Polish National Board of Appeal”, arXiv:2511.04205v1
- General guidelines: https://libguides.gwu.edu/textanalysis
- Social media-related use case: https://www.ibm.com/think/topics/text-mining-use-cases
- Khadse, Dr & Sehgal, Pooja & Sehrawat, Rasna & Nisa, M. & Saranyadevi, M.. (2023). Analyzing Successful Entrepreneurs' Psychological Well-Being, Emotional Health & Mental Disorders. Journal for ReAttach Therapy and Developmental Diversities. 6. 263-275.
- Sentence transformers: https://www.sbert.net/index.html
- HF: https://huggingface.co/models?library=sentence-transformers&author=sentence-transformers
- E. Montahaei, D. Alihosseini, M. Soleymani Baghshah, “Jointly Measuring Diversity and Quality in Text Generation Models”, arXiv:1904.03971

Thank you for Attention <3